KOENIG

step forward



Streams and Lambdas

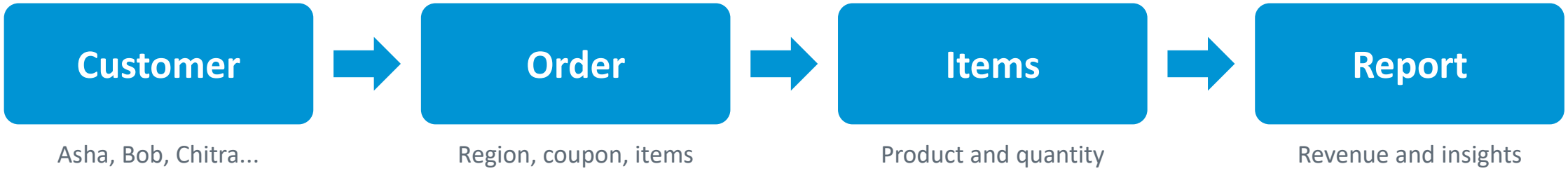
Core Java (Advanced)

Streams and Lambdas

1. Functional interfaces
2. Lambdas, method references, closures
3. Stream pipeline anatomy and laziness
4. map, filter, reduce, flatMap, sorted
5. Collectors: groupingBy, teeing and more
6. Optional: no more null surprises

Demo Scenario: ShopKart Order Analytics

Module 2 stored ShopKart data in collections. Now we ask questions of it with streams. ShopKart has a day of orders. We turn that raw data into answers (revenue, best customers, coupon use) with lambdas and streams instead of loops.



Supporting cast

Product | **Category** | **Coupon** | **Region**

Why Streams?

- **Loops say HOW:** counters, if-blocks, temporary variables.
- **Streams say WHAT:** filter these, transform them, add them up.
- **Less noise:** fewer places for bugs to hide.
- **Analogy:** a factory conveyor belt - items move, machines act on them.

```
List<Integer> numbers = List.of(1, 2, 3, 4, 5, 6);

// Loop: sum of squares of even numbers
int sum = 0;
for (int n : numbers) {
    if (n % 2 == 0) {
        sum += n * n;
    }
}

// Stream: same job, reads like a sentence
int total = numbers.stream()
        .filter(n -> n % 2 == 0)
        .mapToInt(n -> n * n)
        .sum();          // 56
```

Functional Interfaces

- **Definition:** an interface with exactly ONE abstract method.
- **Why it matters:** a lambda is a short way to write that one method.
- **@FunctionalInterface** makes the compiler check the rule.
- **Java ships many** ready-made in java.util.function.

```java
@FunctionalInterface
interface Greeter {
    String greet(String name);   // the ONE method
}

public class Demo {
    public static void main(String[] args) {
        Greeter hello = name -> "Hello, " + name;
        Greeter shout = name -> name.toUpperCase();

        System.out.println(hello.greet("Asha"));
        // Hello, Asha
        System.out.println(shout.greet("Asha"));
        // ASHA
    }
}
```

The Big Five Interfaces

Function<T, R>

apply(T) returns R. Transforms one value into another.

Predicate<T>

test(T) returns boolean. A yes/no question used by filter.

Consumer<T>

accept(T) returns nothing. Does something with a value, like printing.

Supplier<T>

get() takes nothing, returns T. Produces a value on demand.

BiFunction<T, U, R>

apply(T, U) returns R. Like Function but with two inputs.

The Big Five in Code

```
Function<String, Integer> length = s -> s.length();
Predicate<Integer> isEven = n -> n % 2 == 0;
Consumer<String> printer = s -> System.out.println(s);
Supplier<String> greeting = () -> "Hello";
BiFunction<Integer, Integer, Integer> add = (a, b) -> a + b;

length.apply("lambda");      // 6
isEven.test(7);              // false
printer.accept("Hi");        // prints Hi
greeting.get();              // "Hello"
add.apply(3, 4);             // 7
```

- **Combine:** isEven.negate(), p1.and(p2)
- **Chain:** f.andThen(g)
- **Rule:** call the method named in the interface.

The interface name tells you the shape: input type(s) and output type

Demo: Functional Interfaces

What we build

- **Discount rule:** our own @FunctionalInterface, implemented with lambdas.
- **The Big Five:** Predicate, Function, Consumer, Supplier and BiFunction on products.
- **Combining:** and, negate and andThen build bigger rules from small ones.

How the concept helps

- **One method** is all a lambda needs to implement.
- **Ready-made types** mean we rarely write our own interface.
- **Composition** keeps every rule small and reusable.

Lambda Expressions

- **Shape:** (parameters) -> body
- **Types:** Java infers them from the interface.
- **One parameter:** brackets are optional.
- **Block body:** use { } and an explicit return.

```
() -> 42                          // no parameters

x -> x * 2                        // one parameter

(a, b) -> a + b                   // two parameters

(String s) -> s.trim()            // explicit type

(a, b) -> {                       // block body
    int sum = a + b;
    return sum;
}
```

Method References



1 Static method

Integer::parseInt

replaces s -> Integer.parseInt(s)

2 Method on an object

System.out::println

replaces x -> System.out.println(x)

3 Method on the type

String::length

replaces s -> s.length()

4 Constructor

ArrayList::new

replaces () -> new ArrayList<>()

```
names.stream().map(String::toUpperCase).forEach(System.out::println);
```

Use a method reference when the lambda only calls one existing method

Closures: Capturing Variables

- **Closure:** a lambda that remembers variables around it.
- **Rule:** captured locals must be effectively final.
- **Why:** the lambda may run later, on another thread.
- **Fix:** let the stream compute the result.

Broken: lambda changes a local

```
int total = 0;
names.forEach(n -> total += n.length());
// error: local variables referenced from a
// lambda expression must be final or
// effectively final
```

Fixed: return a value instead

```
int total = names.stream()
        .mapToInt(String::length)
        .sum();
```

Demo: Lambdas, Method References and Closures

What we build

- **Lambda shapes:** no, one and two parameters, and a block body.
- **Method references:** static, object, type and constructor forms.
- **Closure error:** a counter changed inside a lambda, then fixed with a stream.

How the concept helps

- **Lambdas** turn behaviour into a value we can pass.
- **Method references** remove noise when a lambda calls one method.
- **Effectively final** keeps lambdas safe when they run later.

Stream Pipeline Anatomy




```
List<String> result = names.stream()      // source
        .filter(n -> n.length() > 3)       // intermediate
        .map(String::toUpperCase)          // intermediate
        .collect(Collectors.toList());     // terminal
// [ASHA, CHEN, DIVYA]
```

- **Sources:** collection.stream(), Stream.of()
- **Intermediate:** return another Stream
- **Terminal:** returns a result and ends the stream

Laziness: Nothing Runs Early

- **Intermediate ops** only describe the plan.
- **The terminal op** starts the belt.
- **Short-circuit:** findFirst, limit, anyMatch stop early.
- **Result:** less work, same answer.

```
Stream<String> plan = names.stream()
    .peek(n -> System.out.println("checking " + n))
    .filter(n -> n.length() > 3);
System.out.println("Plan built");
// Plan built      <- nothing checked yet

plan.findFirst();
// checking Asha   <- stops at first match
```

Without the terminal operation the pipeline is only a recipe.

names = Asha, Ben, Chen, Divya

Demo: Stream Pipeline and Laziness

What we build

- **Loop vs stream:** the same order total written both ways.
- **Pipeline:** source, intermediate steps and a terminal operation.
- **Laziness:** peek shows nothing runs until findFirst starts the belt.

How the concept helps

- **Streams** describe what we want, not how to loop.
- **Intermediate steps** only build a plan.
- **Short-circuit** stops early, so less work gives the same answer.

map and filter

filter keeps items that pass a Predicate

```
List<String> names =
    List.of("Asha", "Ben", "Chen", "Divya");

names.stream()
     .filter(n -> n.length() > 3)
     .toList();
// [Asha, Chen, Divya]
```

map turns each item into something else

```
names.stream()
     .map(String::length)
     .toList();
// [4, 3, 4, 5]

names.stream()
     .map(n -> n.charAt(0))
     .toList();
// [A, B, C, D]
```

filter changes how MANY items; map changes WHAT the items are

reduce: Many Values Into One




```
int sum = List.of(1, 2, 3, 4).stream()
        .reduce(0, (a, b) -> a + b);       // 10

Optional<Integer> max = List.of(3, 9, 4).stream()
        .reduce(Integer::max);             // Optional[9]
```

- **Identity:** the starting value (0 for a sum).
- **Accumulator:** how to combine two values.
- **No identity:** result is an Optional, the stream may be empty.

sorted, distinct, limit

```
List.of(5, 3, 5, 1, 3, 4).stream()
    .distinct()     // 5, 3, 1, 4
    .sorted()       // 1, 3, 4, 5
    .limit(3)       // 1, 3, 4
    .toList();      // [1, 3, 4]

names.stream()
    .sorted(Comparator.comparing(
            String::length).reversed())
    .toList();      // [Divya, Asha, Chen, Ben]
```

distinct

Removes duplicates using equals().

sorted

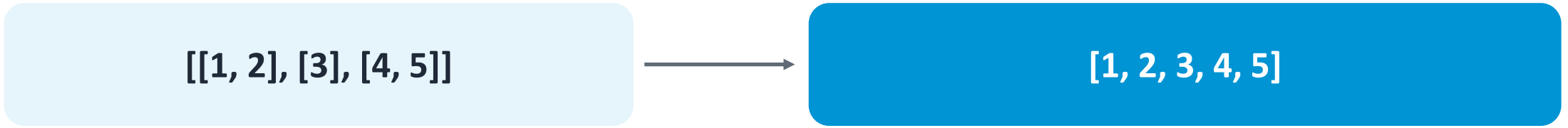
Natural order, or pass a Comparator.

limit

Keeps only the first n items.

Order matters: distinct and limit BEFORE an expensive step saves work

flatMap: Flatten Nested Data




```
List<List<Integer>> nested =
    List.of(List.of(1, 2), List.of(3), List.of(4, 5));

nested.stream().map(List::size).toList();      // [2, 1, 2]

nested.stream()
      .flatMap(list -> list.stream())          // list -> Stream
      .toList();                               // [1, 2, 3, 4, 5]
```

- **map:** one item in, one item out.
- **flatMap:** one item in, a STREAM out - then all streams are merged.
- **Typical use:** Order -> its OrderItems.

Demo: map, filter, reduce, flatMap and sorted

What we build

- **Products:** filter by price, map to names and lengths.
- **Orders to items:** flatMap turns a list of orders into one list of items.
- **Totals and ranking:** reduce for revenue (as numbers and as Money), sorted, distinct and limit for the top 3.

How the concept helps

- **filter** changes how many items; map changes what they are.
- **flatMap** flattens nested lists into one stream.
- **reduce** folds many values into one answer.

Collectors: toList and joining

- **collect()** is the terminal op that builds a result.
- **Collectors** are ready-made recipes for it.
- **toList:** gather items into a List.
- **joining:** glue strings into one String.

```
names.stream().collect(Collectors.toList());
// [Asha, Ben, Chen, Divya]

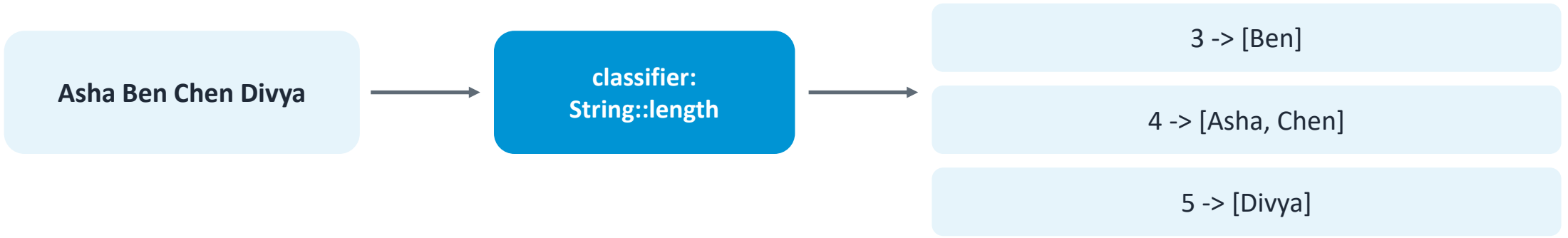
names.stream().collect(Collectors.joining(", "));
// Asha, Ben, Chen, Divya

names.stream()
     .collect(Collectors.joining(" | ", "<", ">"));
// <Asha | Ben | Chen | Divya>
```

joining needs a Stream of String - map to String first

groupingBy: Sort Into Piles

```
Map<Integer, List<String>> byLength =
    names.stream()
         .collect(Collectors.groupingBy(String::length));
// {3=[Ben], 4=[Asha, Chen], 5=[Divya]}
```



- **Classifier:** a Function that picks the group (the key).
- **Result:** a Map; each value is a List by default.

Downstream Collectors

- **Downstream:** a second collector that works on each pile.
- **counting, summingDouble,** mapping, averagingInt ...
- **Nest groupingBy** for two-level summaries.

```
record Sale(String region, String category,
            double amount) {}

// sales: North/Electronics 1200, North/Furniture 200,
//        South/Furniture 300

sales.stream().collect(groupingBy(Sale::region,
        summingDouble(Sale::amount)));
// {South=300.0, North=1400.0}

sales.stream().collect(groupingBy(Sale::region,
        groupingBy(Sale::category,
                   summingDouble(Sale::amount))));
// {South={Furniture=300.0},
//  North={Electronics=1200.0, Furniture=200.0}}
```

partitioningBy and Summarizing

```
List<Integer> nums = List.of(1, 2, 3, 4, 5, 6);

nums.stream().collect(
    Collectors.partitioningBy(n -> n % 2 == 0));
// {false=[1, 3, 5], true=[2, 4, 6]}

IntSummaryStatistics stats = nums.stream()
    .collect(Collectors.summarizingInt(n -> n));
// IntSummaryStatistics{count=6, sum=21, min=1,
//   average=3.500000, max=6}
stats.getMax();       // 6
```

partitioningBy

Exactly two groups: true and false. Both keys always exist.

summarizing

count, sum, min, max and average in one pass.

teeing: Two Results in One Pass




```
double average = nums.stream().collect(Collectors.teeing(
        Collectors.summingInt(n -> n),     // 21
        Collectors.counting(),             // 6
        (sum, count) -> sum / (double) count));
// 3.5
```

Use teeing when you need two answers but can read the stream only once

Demo: Collectors and Summaries

What we build

- **Grouping:** orders by region, with counts and revenue as downstream collectors.
- **Two levels:** region, then category, over flattened sales.
- **Statistics:** partitioningBy, summarizingDouble and teeing for the average basket.

How the concept helps

- **groupingBy** sorts data into piles in one line.
- **Downstream collectors** summarise each pile.
- **teeing** gives two answers from one pass over the data.

Optional: A Box That May Be Empty

- **Problem:** null hides in return types and explodes later.
- **Optional<T>** says out loud: the value may be missing.
- **Analogy:** a lunchbox - open it before you eat.
- **Use it for** return types, not for fields or parameters.

```
Optional<String> a = Optional.of("Asha");
// value must NOT be null

Optional<String> b = Optional.ofNullable(maybeNull);
// empty if maybeNull is null

Optional<String> c = Optional.empty();
// explicitly nothing

a.isPresent();      // true
c.isEmpty();        // true
```

Chaining Optional

```
Optional<String> email = Optional.of("  Asha@Example.com ");

String domain = email
        .map(String::trim)                      // clean up
        .filter(e -> e.contains("@"))           // validate
        .map(e -> e.substring(e.indexOf('@') + 1))
        .orElse("none");
// Example.com
```

- **map:** transform the value if present.
- **filter:** empty the box if the test fails.
- **flatMap:** when the function itself returns an Optional.

Same words as streams: map and filter - an Optional is a stream of 0 or 1

Unwrapping Optional Safely

Safe

```
opt.orElse("default");
opt.orElseGet(() -> compute());
opt.orElseThrow(() ->
    new NoSuchElementException("missing"));
opt.ifPresent(v -> System.out.println(v));
opt.ifPresentOrElse(
    v -> use(v), () -> warn());
```

Risky

```
opt.get();
// NoSuchElementException if empty

if (opt.isPresent()) {
    use(opt.get());   // just null-check again
}
```

- **orElse** evaluates its argument always; orElseGet only when empty.

Default value, throw, or consume - always decide what 'empty' means

Demo: Optional

What we build

- **Coupons:** an order may or may not have one, and the type says so.
- **Chains:** map, filter and flatMap turn a code into a discount.
- **Unwrapping:** orElse vs orElseGet, and ifPresentOrElse.

How the concept helps

- **Optional** makes a missing value visible in the code.
- **Chaining** skips every step when the box is empty.
- **orElseGet** avoids work we do not need.

Key Takeaways

1. **Lambdas**
   Short functions that implement a functional interface.

2. **Pipelines**
   Source, intermediate ops, terminal op - lazy until the end.

3. **Collectors**
   groupingBy, partitioningBy and teeing summarise data.

4. **Optional**
   Replace null with Optional



Happy Learning :)